

复杂数据类型



北京理工大学

每个数据都有一个类型，我们可以使用内置的 `type` 函数返回任何表达式的计算结果所属的数据类型。先看几个例子。

```
type(5)
```

`int`

```
type(5.)
```

`float`

```
type(5.0)
```

`float`

浮点数和整数的区别之一就是浮点数有小数点，但整数没有小数点。

```
type(5/1)
```

float

注意，在上面的例子中，type 函数先计算表达式的最终值，返回最终值的类型。两个整数相除，即使能够整除，结果一定是浮点数。

```
type(5//2)
```

int

```
type(2+2j)
```

complex

```
type('5')
```

str

接上页

```
type(['A','B'])
```

list

```
type(('A','B'))
```

tuple

```
type({'Name':'A',B'})
```

dict

列表 (list)、元组 (tuple)、字典 (dict) 是我们本章需要学习的重点内容。另外，内置函数和方法被视为是一种数据类型，是 `builtin_function_or_method`。下面再给出一些内置函数的例子。

```
type(abs)
```

```
builtin_function_or_method
```

```
type(max)
```

```
builtin_function_or_method
```

```
type(len)
```

```
builtin_function_or_method
```

列表

在前一章，我们已经介绍过数值型数据，比如整数、浮点数、复数等。我们也介绍过字符串。这一章会探索一些比较复杂的数据类型。我们重点介绍列表、元组、字典和数组四种复杂数据类型。

列表是 Python 中非常基本的数据结构，支持字符、数字、字符串、列表 (即嵌套)。列表的数据项不需要具有相同的类型。列表用 `[]` 标识。列表是一种有序的集合，可以随时添加和删除其中的元素。列表中每个元素都分配一个索引，第一个索引是 0，第二个索引是 1，以此类推。切割列表中的值时，用 [头下标: 尾下标]。Python 列表有 2 种取值顺序：头下标从左到右索引默认以 0 开始，尾下标最大范围是字符串长度少 1；头下标从右到左索引默认以 -1 开始，尾下标最大范围是字符串开头。头下标和尾下标都可以为空表示取到头或尾。

```
mylist=['LEE',123,3.1,'LILY']  
mylist
```

```
['LEE', 123, 3.1, 'LILY']
```

用索引从左到右来访问 list 中每一个位置的元素，索引是从 0 开始的：

```
mylist[0]
```

'LEE'

```
mylist[1]
```

123

```
mylist[2]
```

3.1

```
mylist[3]
```

'LILY'

当索引超出了范围时，Python 会报一个 `IndexError` 错误，所以，要确保索引不要越界，最后一个元素的索引是 `len(list)-1`。如果要取最后一个元素，除了计算索引位置外，还可以用 `-1` 做索引，直接获取最后一个元素：

```
mylist[-1]
```

'LILY'

以此类推，可以获取倒数第 2 个、倒数第 3 个：

```
mylist[-2]
```

3.1

```
mylist[-3]
```

123

```
mylist[0][2]
```

'P'

输出第 2 个至第 3 个元素

```
mylist[1:3]
```

```
[123, 3.1]
```

输出第 3 个至列表末尾的所有元素

```
mylist[2:]
```

```
[3.1, 'LILY']
```

```
my_family=['爷爷','奶奶','爸爸','妈妈','我']  
my_family
```

```
['爷爷','奶奶','爸爸','妈妈','我']
```

```
len(my_family)
```

接上页

```
my_family[0]
```

'爷爷'

```
my_family[-1]
```

'我'

```
my_family[1:4]
```

['奶奶', '爸爸', '妈妈']

```
my_family[-4:-1]
```

['奶奶', '爸爸', '妈妈']

对列表进行操作时，* 表示列表被重复多次。比如说，

```
my_family*2
```

```
['爷爷','奶奶','爸爸','妈妈','我','爷爷','奶奶','爸爸','妈妈','我']
```

对列表进行操作时，+ 表示两个列表组合。

```
'数据'+ '科学'+ '基础'
```

```
'数据科学基础'
```

```
tinylis = [ 123, '321']  
comblis=mylis+ tinylis  
comblis
```

['LEE', 123, 3.1, 'LILY', 123, '321']

把上面的结果与下面的运行结果比较一下

```
newlis=[mylis, tinylis ]  
newlis
```

[['LEE', 123, 3.1, 'LILY'], [123, '321']]

```
comblis[0]
```

'LEE'

```
newlis[0]
```

['LEE', 123, 3.1, 'LILY']

变量 `list` 就是一个列表。用 `len()` 函数可以获得 `list` 元素的个数：

```
len(mylist)
```

4

如果 `mylist` 中一个元素也没有，就是一个空的 `list`，它的长度为 0：

```
len([])
```

0

注意，mylist 是一个可变的有序表，所以，可以往 mylist 中追加元素到末尾：

```
mylist.append('DUDU')  
mylist
```

```
['LEE', 123, 3.1, 'LILY', 'DUDU']
```

也可以把元素插入到指定的位置，比如索引号为 1 的位置：

```
mylist.insert(1, 'flower')  
mylist
```

```
['LEE', 'flower', 123, 3.1, 'LILY', 'DUDU']
```

使用 del 语句来删除列表的元素。

```
del mylist[1]  
mylist
```

```
['LEE', 123, 3.1, 'LILY', 'DUDU']
```

要删除 mylist 末尾的元素，用 pop() 方法：

```
mylist.pop()  
mylist
```

`['LEE', 123, 3.1, 'LILY']`

要删除指定位置的元素，用 pop(i) 方法，其中 i 是索引位置：

```
mylist.pop(1)  
mylist
```

`['LEE', 3.1, 'LILY']`

要把某个元素替换成别的元素，可以直接赋值给对应的索引位置：

```
mylist[1]=2+1j  
mylist
```

`['LEE', (2+1j), 'LILY']`

上面的例子同时也暗示 mylist 里面的元素的数据类型也可以不同。

下面给出一些常用的列表操作，比如 +、* 等运算符、成员运算符等。

Python 表达式	结果	描述
[1,2,3]+[4,5,6]	[1,2,3,4,5,6]	组合
['Hi!']*4	['Hi!','Hi!','Hi!','Hi!']	重复
3 in [1,2,3]	True	元素是否存在于列表中
for x in [1,2,3]: print(x),	1,2,3	迭代

列表可以作为函数的参数，Python 包含 `cmp`，`len`，`max`，`min` 以及 `list` 等函数，这个函数的功能列在下面的表格中。

函数	描述
<code>cmp(list1,list2)</code>	比较两个列表的元素
<code>len(list)</code>	列表元素个数
<code>max(list)</code>	返回列表元素最大值
<code>min(list)</code>	返回列表元素最小值
<code>list(seq)</code>	将元组转换为列表

对列表进行操作，Python 有如下一些方法。

方法	描述
<code>list.append(obj)</code>	在列表末尾添加新的对象
<code>list.count(obj)</code>	统计某个元素在列表中出现的次数
<code>list.extend(seq)</code>	在列表末尾一次性追加另一个序列中的多个值 (用新列表扩展原来的列表)
<code>list.index(obj)</code>	从列表中找出某个值第一个匹配项的索引位置
<code>list.insert(index,obj)</code>	将对象插入列表
<code>list.pop(index=-1)</code>	将移除列表中的一个元素 (默认最后一个元素)，并且返回该元素的值
<code>list.remove(obj)</code>	移除列表中某个值的第一个匹配项
<code>list.reverse()</code>	反向列表中元素
<code>list.sort(cmp=None, key=None,reverse=False)</code>	对原列表进行排序

有三个方法与删除操作有关系：remove 移除的是列表中首个符合条件的元素，并不删除特定的索引；del 删除是按索引来的，索引起始位置为 0；pop 和 del 是比较接近，都是按照索引删除字符，但 pop 的返回值可以赋值给其他的变量，返回的是删除的那个数值。

```
a=[1,2,3,5,4,2,6]
```

```
del a[5]
```

```
a
```

```
[1, 2, 3, 5, 4, 6]
```

```
a=[1,2,3,5,4,2,6]
```

```
a.remove(a[5])
```

```
a
```

```
[1, 3, 5, 4, 2, 6]
```

```
n =[1,2,2,3,4,5]
```

```
n.remove(3)
```

```
print(n)
```

```
[1, 2, 2, 4, 5]
```

接下来做一个小练习，取出下面这个矩阵的对角线元素。

```
L = [  
    ['A', 'B', 'C', 'D'],  
    ['E', 'F', 'G', 'H'],  
    ['I', 'J', 'K', 'L'],  
    ['M', 'N', 'O', 'P'],  
]  
L[0][0]
```

'A'

```
L[2][2]
```

'K'

```
D = L[0][0] + L[1][1] + L[2][2] + L[3][3]  
D
```

'AFKP'

试试看能不能用 `D.insert(0,'HHHH')`?

```
D.insert(0, 'HHHH')
```

```
D
```

元组也是 Python 中经常使用的数据类型。元组用圆括号 () 标识，列表用方括号 [] 标识。元组创建很简单，只需要在括号中添加元素，并使用逗号隔开即可。元组是一种有序的集合，和列表非常类似，但是元组一旦初始化就不能修改。因此，元组没有 append()、insert() 这样的方法。其他获取元素的方法和列表是一样的，但不能赋值成另外的元素。

```
mytuple= ('LEE', 123 , 3.1, 'LILY')  
mytuple
```

```
('LEE', 123, 3.1, 'LILY')
```

如果要定义一个空的 tuple, 可以写成 ():

```
null = ()  
len(null)
```

0

但是, 要定义一个只有 1 个元素的 tuple, 如果你这么定义,

```
ten = (10)  
ten
```

10

```
type(ten)
```

int

刚才定义的并不是 tuple, 而是 10 这个数字! 这是因为, 圆括号 () 既可以表示 tuple, 又可以表示数学公式中的小括号, 这就产生了歧义。

因此，Python 规定，这种情况下，按小括号进行计算，计算结果自然是 10。只有一个元素的 tuple 定义时必须加一个逗号，来消除歧义，以免误解成数学计算意义上的小括号。

```
ten = (10,)
ten
```

```
(10,)
```

```
type(ten)
```

tuple

刚才这个例子稍微复杂一点，已经牵涉到我们接下来要介绍的字典了。

字典是一种非常灵活的可变容器模型，可存储任意类型对象。字典和列表或元组至少有两个区别：一是列表或元组是有序的对象集合，字典是无序的对象集合。二是字典当中的元素是通过键而不是位置来提取的。字典用花括号 `{}` 识别。字典由索引 (key) 和它对应的值 value 组成。每个键与对应的值对用冒号分割，每个键值对之间用逗号分割，整个字典包括在花括号 `{}` 中。标准格式如下：`d = key1:value1, key2:value2`。键一般是唯一的，如果重复定义键，最后出现的键的值会自动替换前面的。值不需要唯一。

```
mydict = {'Name': 'LEE'}  
mydict
```

```
{'Name': 'LEE'}
```

```
mydict = {'Name': ['LEE', 'Lily']}  
mydict
```

```
{'Name': ['LEE', 'Lily']}
```

字典是通过键提取元素的

```
mydict['Name']
```

```
['LEE', 'Lily']
```

```
mydict['Name'][0]
```

```
'LEE'
```

```
dict={'统计学': ['数理统计', '经济统计', '应用统计']}
```

```
dict['统计学'][1]
```

```
'经济统计'
```

```
dict={'统计学': ['数理统计', '应用统计'], '数学': ['应用数学', '基础数学']}
```

```
dict['数学']
```

```
['应用数学', '基础数学']
```

```
dict['统计学']
```

```
['数理统计', '应用统计']
```

键必须不可变，可以用数字，字符串或元组充当，但键不可以用列表：

```
mydict = { }  
mydict['one'] = "This is one"  
mydict[2] = "This is two"  
mydict
```

```
{2: 'This is two', 'one': 'This is one'}
```

要求输出键为 2 的值，并不是第 2 个位置的元素！

```
mydict[2]
```

```
'This is two'
```

```
mydict['one']
```

```
'This is one'
```

我们可以输出字典的键和值：

```
dict.keys()
```

```
dict_keys(['one', 2])
```

```
dict.values()
```

```
dict_values(['This is one', 'This is two'])
```

修改键值的操作如下：

```
dict = {'姓名': '小朱', '学历': '博士', '性别': '男'};
```

```
dict['姓名'] = '老朱';
```

```
dict['学历'] = "研究生";
```

```
dict['姓名']
```

```
'老朱'
```

```
dict['学历']
```

```
'研究生'
```

字典 (dict) 删除元素, 可以选择两种方式, `dict.pop(key)` 和 `del dict[key]`。

```
dict = {'姓名': '小朱', '学历': '博士', '性别': '男'};
del dict['性别']
dict
```

{' 姓名': ' 小朱', ' 学历': ' 博士'}

```
dict = {'姓名': '小朱', '学历': '博士', '性别': '男'};
dict.pop('性别')
dict
```

{' 姓名': ' 小朱', ' 学历': ' 博士'}

清空词典所有条目，用于清空 (或删除) 字典中的所有数据项。

```
dict = {'姓名': '小朱', '学历': '博士', '性别': '男'};
dict.clear()
dict
```

{}

也可以直接删除整个词典，

```
dict = {'姓名': '小朱', '学历': '博士', '性别': '男'};
del dict
dict
```

下面给出字典的一些常用操作，这些操作既有方法，也有函数。

函数	描述
<code>cmp(dict1,dict2)</code>	比较两个字典元素。
<code>len(dict)</code>	计算字典元素个数，即键的总数。
<code>str(dict)</code>	输出字典可打印的字符串表示。
<code>type(variable)</code>	返回输入的变量类型，如果变量是字典就返回字典类型。
<code>dict.clear()</code>	删除字典内所有元素。
<code>dict.copy()</code>	返回一个字典的浅复制。
<code>dict.fromkeys(seq[,val])</code>	创建一个新字典，以序列 <code>seq</code> 中元素做字典的键， <code>val</code> 为字典所有键对应的初始值。
<code>dict.get(key,default=None)</code>	返回指定键的值，如果值不在字典中返回 <code>default</code> 值。
<code>dict.has_key(key)</code>	如果键在字典 <code>dict</code> 里返回 <code>true</code> ，否则返回 <code>false</code> 。
<code>dict.items()</code>	以列表返回可遍历的 (键, 值) 元组数组。
<code>dict.keys()</code>	以列表返回一个字典所有的键。
<code>dict.setdefault(key,default=None)</code>	和 <code>get()</code> 类似，但如果键不存在于字典中，将会添加键并将值设为 <code>default</code> 。
<code>dict.update(dict2)</code>	把字典 <code>dict2</code> 的键/值对更新到 <code>dict</code> 里。
<code>dict.values()</code>	以列表返回字典中的所有值。
<code>pop(key[,default])</code>	删除字典给定键 <code>key</code> 所对应的值，返回值为被删除的值。 <code>key</code> 值必须给出。否则，返回 <code>default</code> 值。
<code>popitem()</code>	随机返回并删除字典中的一对键和值。

元组的一级元素不可被修改增加删除，但是可以修改二级后的列表或者字典。

```
tu = ("alex", [11, 22, {"k1": 'v1', "k2": ["v2", "v3"]}, 44])  
tu [1][2][ "k2"].append("v4")  
tu [1][2][ "k2"]
```

['v2', 'v3', 'v4']

数组的概念与线性代数中的向量比较接近，但是，数组可以包含字符串。数字也可以包含其他类型的数值数据，但是单个数组只能包含单一类型的数据。这是和之前提到的列表、元组不一样的地方。在介绍数组之前，我们先定义 `make_array` 函数创建数组。

```
import numpy as np  
make_array = lambda *args: np.asarray(args)
```

事实上，这个函数与直接使用调用函数 `np.asarray()` 是一样的。

数组可以包含字符串或其他类型的值，但是单个序列只能包含单一类型的数据。例

如：

```
baseline_high = 14.48  
highs = make_array(baseline_high - 0.880, baseline_high - 0.093,  
baseline_high + 0.105, baseline_high + 0.684)  
highs
```

```
array([13.6 , 14.387, 14.585, 15.164])
```

我们可以尝试一下如下的程序，看看会出现什么结果。

```
baseline_high = 14.48  
highs = baseline_high + make_array( -0.880, -0.093, 0.105, 0.684)  
highs
```

注意，`baseline_high` 中间有一根小的下划线，而不是一个空格。

```
highs[1]
```

```
14.387
```

数组允许我们使用单个名称，将多个值传递个一个函数。例如，sum 函数计算序列中所有值的和，len 函数计算其长度。一起使用它们，我们可以计算一个集合的平均值。

```
sum(highs)/len(highs)
```

```
14.434000000000001
```

数组可以用在算术表达式中来计算其内容。当数组与单个数组合时，该数与数组的每个元素组合。因此，我们可以通过编写熟悉的转换公式，将所有这些温度转换成华氏温度。

```
(9/5) * highs + 32
```

```
array([56.48 , 57.8966, 58.253 , 59.2952])
```

要看 tuple 和数组之间的区别，对比一下下面的函数就可以知道了。

```
baseline_high = 14.48
highs_tuple = (baseline_high - 0.880, baseline_high - 0.093,
baseline_high + 0.105, baseline_high + 0.684)
(9/5) * highs_tuple[0] + 32
(9/5) * highs_tuple[1] + 32
(9/5) * highs_tuple[2] + 32
(9/5) * highs_tuple[3] + 32
```

下面这个命令时能不能执行的呢？比如

```
(9/5) * highs_tuple + 32,sum(highs_tuple)
```

我们经常可能需要对多个数组进行操作。如果两个数组是一样大小，我们可以很容易对两个不同的数组进行计算操作。我们再来看看之前的例子：

```
baseline_low = 3.00  
lows = baseline_low + make_array( - 0.872, - 0.629, - 0.126, 0.728)  
lows
```

```
array([2.128, 2.371, 2.874, 3.728])
```

要看最高温度和最低温度的差值，比较麻烦的做法是这样的：

```
make_array(highs[0] - lows[0], highs[1] - lows[1], highs[2] - lows[2], highs[3] - lows[3])
```

```
array([11.472, 12.016, 11.711, 11.436])
```

但是，如果使用数组来看看最高温度和最低温度的差值的话，我们就不需要这么麻烦了。至于如何做，我们就留给大家作为课后作业了。

数组也有方法，这些方法是操作数组值的函数。数值集合的均值是其总和除以长度。以下示例中的每对括号都是调用表达式的一部分；它调用一个无参数函数来对数组 `highs` 进行计算。

```
highs.size
```

4

```
highs.size
```

57.736000000000004

```
highs.mean()
```

14.434000000000001

对数组进行操作，在 numpy 模块中为程序员提供了许多创建和操作数组的函数。例如，diff 函数计算数组中每两个相邻元素之间的差。差数组的第一个元素是原数组的第二个元素减去第一个元素。

```
np.diff(highs)
```

```
array([0.787, 0.198, 0.579])
```

关于 Numpy 模块，如果需要非常完整、系统地学习操作手册，大家可以参考 <https://docs.scipy.org/doc/numpy/reference/>。这个操作手册详细列出了这些功能，学习这些操作手册尽管是学习 Python 语言的重要组成部分，但是，你并不需要记住全部命令，只需要知道 Python 数组操作有这个功能即可。在真正需要的时候，可以返回来查询这些操作手册。

下面我们给出一些数组的常用操作：

函数	描述
np.prod	将所有元素相乘
np.sum	将所有元素相加
np.all	检验所有元素是否都为真值 (或都为非 0 元素)
np.any	检验是否至少有一个元素为真值 (或至少有一个非 0 元素)
np.count_nonzero	计算非 0 元素的个数
np.diff	相邻元素做差
np.round	将每个元素四舍五入到最接近的整数
np.cumprod	累乘：将每个元素乘到一起
np.cumsum	累加：将每个元素加到一起。
np.exp	返回每个元素的 e 的幂次方
np.log	返回每个元素的以 e 为底的对数函数值。
np.sqrt	返回每个元素的开方值。
np.sort	对元素进行排序。

字符串也可以当成数组，调用 `numpy.char` 下的函数进行操作。

函数	描述
<code>np.char.lower</code>	小写每个元素
<code>np.char.upper</code>	大写每个元素
<code>np.char.strip</code>	删除每个元素开头或结尾的空格
<code>np.char.isalpha</code>	判断每个元素是否都是字母 (没有数字或符号)
<code>np.char.isnumeric</code>	判断每个元素是否都是数字 (没有字母)
<code>np.char.count</code>	计算被搜索的字符串出现在数组中的次数
<code>np.char.find</code>	每个元素中被搜索字符串第一次出现的位置
<code>np.char.rfind</code>	每个元素中被搜索字符串最后一次出现的位置
<code>np.char.startswith</code>	每个元素是否以被搜索的字符串开头

下面我们以“范围”这个函数为例子，来说明数组的使用方法。“范围”函数产生一个数组，按照递增或递减的顺序排列，每个元素按照一定的间隔分开。“范围”函数在很多情况下非常有用，所以值得我们非常深入地了解这个函数。“范围”使用 `np.arange` 函数来定义，该函数接受一个、两个或三个参数：起始值，终止值和步长。如果将一个参数传递给 `np.arange`，那么这个参数被默认为是终止值。注意，终止值不会达到，产生的数组在终止值之前结束。另外，默认的起始值 `start=0`，步长 `step=1`。如果将两个参数传递给 `np.arange`，那么这两个参数分别被默认为是起始值和终止值，注意，终止值不会达到，产生的数组在终止值之前结束。另外，默认的步长 `step=1`。如果将三个参数传递给 `np.arange`，那么这三个参数分别被默认为是起始值，终止值和步长。“范围”函数始终包含起始 `start` 值，但不包括其 `end` 值。它按照 `step` 计数，并在到达终止值 `end` 之前停止。下面的例子中，数值从 0 起始，并仅仅增加到 4，并不是 5。

```
np.arange(5)
```

```
array([0, 1, 2, 3, 4])
```

```
np.arange(3, 9)
```

```
array([3, 4, 5, 6, 7, 8])
```

这个数组从 3 起始，增加了步长 5 变成 8，然后增加步长 5 变成 13，以此类推。

```
np.arange(3, 30, 5)
```

```
array([ 3, 8, 13, 18, 23, 28])
```

当你指定步长时，起始值、终止值和步长可正可负，可以是整数也可以是分数。

```
np.arange(1.5, -2, -0.5)
```

```
array([ 1.5, 1. , 0.5, 0. , -0.5, -1. , -1.5])
```

针对数组进行操作的“范围”函数非常有用，下面我们通过两个计算 π 的数学公式来说明这个函数的用途。首先来看莱布尼茨的 π 公式。

伟大的德国数学家和哲学家戈特弗里德·威廉·莱布尼茨 (Gottfried Wilhelm Leibniz, 1646~1716) 发现了一个简单分数的无穷和。公式是：

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \cdots \right) \quad (1)$$

让我们计算莱布尼茨公式无穷和的前一百万项，看是否接近 π 。我们将计算这个有限的总和，首先加上所有的正项，然后减去所有负项的和：

$$\pi = 4 \left(1 + \frac{1}{5} + \frac{1}{9} + \frac{1}{13} + \cdots \right) - 4 \left(\frac{1}{3} + \frac{1}{7} + \frac{1}{11} + \cdots \right) \quad (2)$$

令人惊讶的是，当我们将无限多个分数相加时，顺序可能很重要。但是我们对 π 的近似只使用了大量的数量有限的分数，所以可以按照任何方便的顺序，将这些项相加。和中的正项分母是 1, 5, 9, 以此类推。数组 `by_four_to_20` 包含 17 之前的这些数。

```
by_four_to_20 = np.arange(1, 20, 4)
by_four_to_20
```

```
array([ 1, 5, 9, 13, 17])
```

为了获得 π 的准确近似，我们使用更长的数组 `positive_term_denominators`。

```
positive_term_denominators = np.arange(1, 1e6, 4)
positive_term_denominators
```

```
array([1.00000e+00, 5.00000e+00, 9.00000e+00, ..., 9.99989e+05, 9.99993e+05,
       9.99997e+05])
```

我们实际打算加起来的正项，就是一除以这些分母。

```
positive_terms = 1 / positive_term_denominators
positive_terms
```

```
array([1.00000000e+00, 2.00000000e-01, 1.11111111e-01, ..., 1.00001100e-06,
       1.00000700e-06, 1.00000300e-06])
```

负项的分母是 3, 7, 11, 以此类推。这个数组可以通过正项数组加上 2 得到。

```
negative_terms = 1 / (positive_term_denominators + 2)
```

整体的和是：

```
4 * (sum(positive_terms) - sum(negative_terms))
```

3.1415906535890485

这非常接近于 $\pi = 3.14159\dots$ 。可以看出，莱布尼茨公式看起来不错。

我们再来看第二个计算 π 的公式。英国数学家 John Wallis(1616~1703) 提出了一个计算 π 的公式

$$\pi = 2 \cdot \left(\frac{2}{1} \cdot \frac{2}{3} \cdot \frac{4}{3} \cdot \frac{4}{5} \cdot \frac{6}{5} \cdot \frac{6}{7} \cdots \right) \quad (3)$$

我们对上述的 π 计算公式调整以下计算顺序：

$$\pi = 2 \cdot \left(\frac{2}{1} \cdot \frac{4}{3} \cdot \frac{6}{5} \cdots \frac{10^6}{999999} \right) \left(\frac{2}{3} \cdot \frac{4}{5} \cdot \frac{6}{7} \cdots \frac{10^6}{10^6 + 1} \right) \quad (4)$$

```
even = np.arange(2,1000001,2)
one_below_even = even - 1
one_above_even = even + 1
2 * np.prod(even/one_below_even)* np.prod(even/one_above_even)
```

3.1415910827951143

Wallis 公式非常准确，小数点的前 5 位都是正确的。